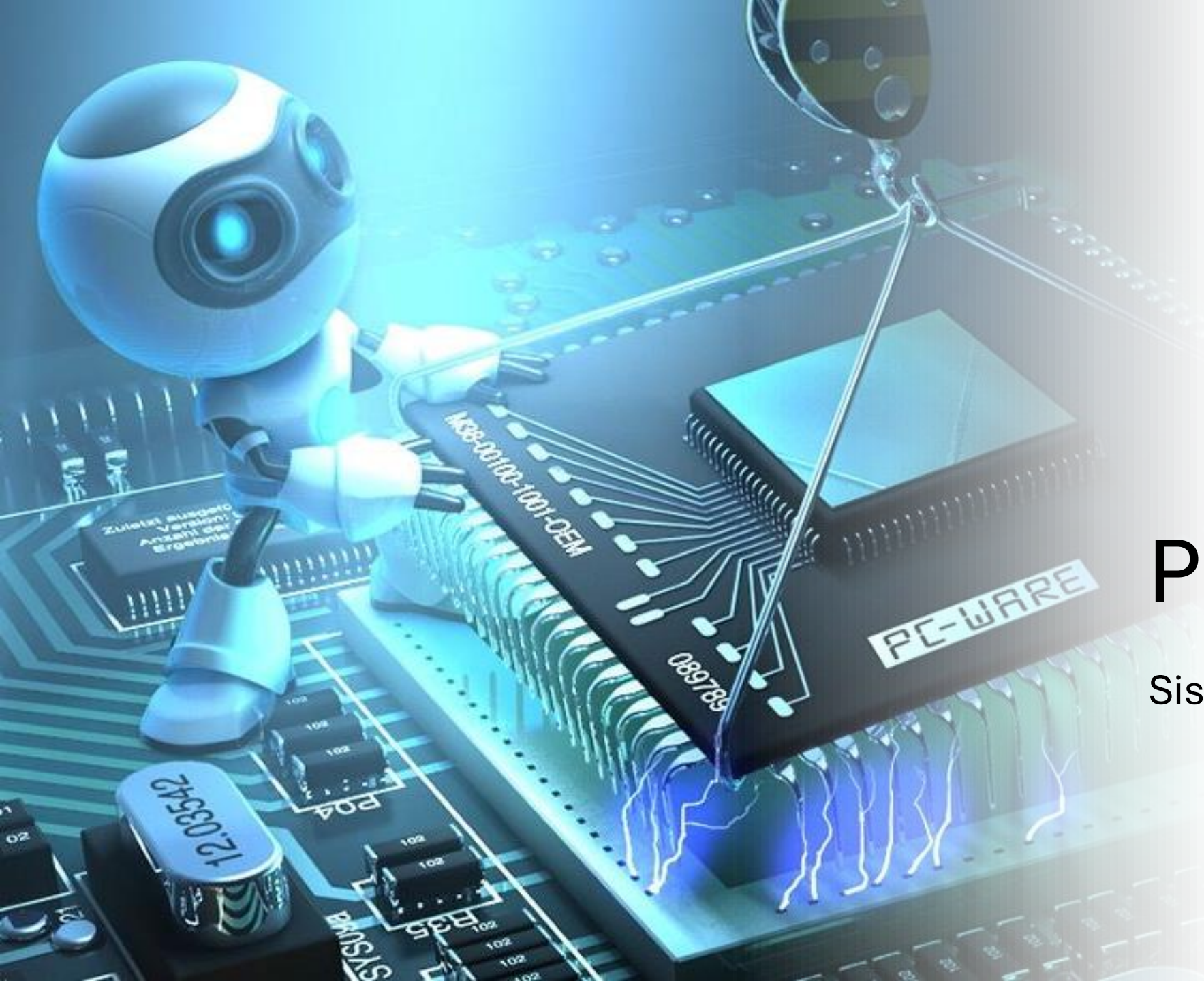




Procesos

Sistemas Operativos

Proceso

- Una abstracción de un programa en ejecución
- Con la multiprogramación es posible tener **pseudoparalelismo** en la ejecución de procesos
- → La CPU conmuta entre programas cada cierto tiempo y de acuerdo a las **políticas de prioridad** implementadas
- Con **multiprocesadores** es posible tener **paralelismo real**.



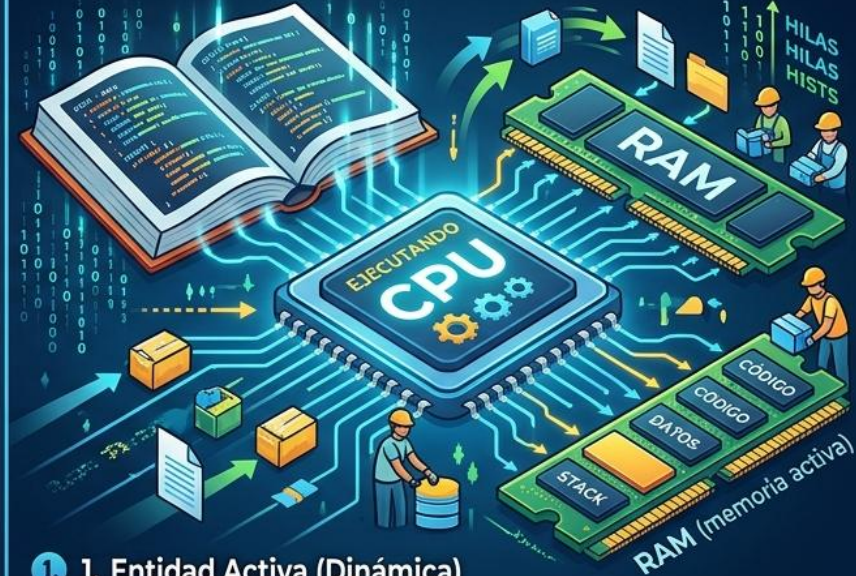
DIFERENCIA: PROGRAMA vs. PROCESO

EL PROGRAMA (ESTÁTICO)



- 1. Entidad Pasiva (Estática)
- 2. Conjunto de instrucciones y datos
- 3. Almacenado en disco (DD/SSD)
- 4. No utiliza CPU ni RAM
- 5. No tiene estado dinámico

EL PROCESO (ACTIVO)



- 1. Entidad Activa (Dinámica)
- 2. Programa en ejecución
- 3. Cargado en memoria RAM
- 4. Utiliza CPU, RAM y recursos
- 5. Tiene ESTADO (Ejecutando, Esperando, etc.)

Proceso vs Programa

El Programa (Entidad Pasiva)

- **¿Qué es?** Archivo binario estático almacenado en disco.
- **Estado:** No tiene ciclo de vida ni estado dinámico.
- **Recursos:** Consume **0%** de CPU y **0%** de memoria RAM.
- **Analogía:** La **receta de cocina** escrita en un libro.



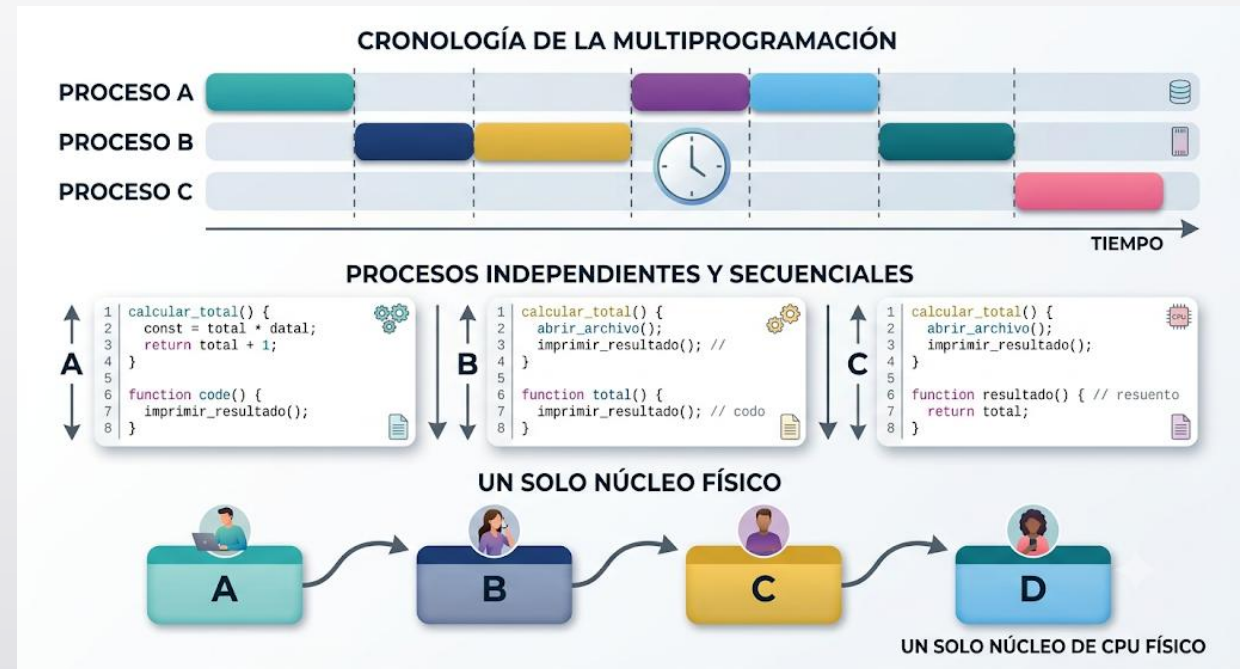
El Proceso (Entidad Activa)

- **¿Qué es?** Abstracción de un programa en plena ejecución.
- **Estado:** Dinámico y mutable, gestionado por el Kernel (*Listo, Ejecutando, Bloqueado*).
- **Recursos:** Asigna registros (Contador de Programa) y espacio operativo en RAM (*Datos, Stack, Heap*).
- **Analogía:** La actividad de cocinar usando los ingredientes.



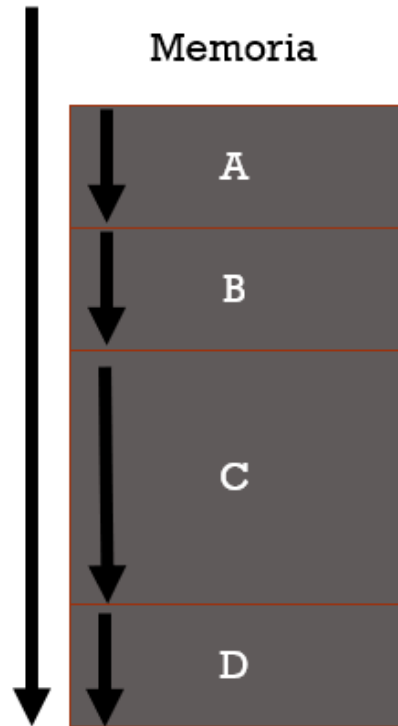
El modelo del proceso

- Considerando **una** sola CPU
- Cada proceso tiene su propio flujo de ejecución.
- Cada proceso posee una CPU virtual.
- El sistema operativo alterna la CPU entre procesos.
- En una sola CPU sólo un proceso puede ejecutarse a la vez.



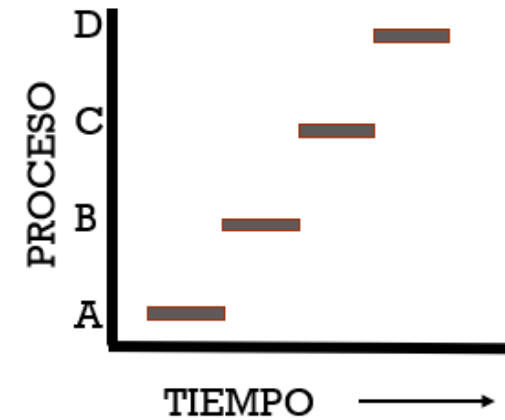
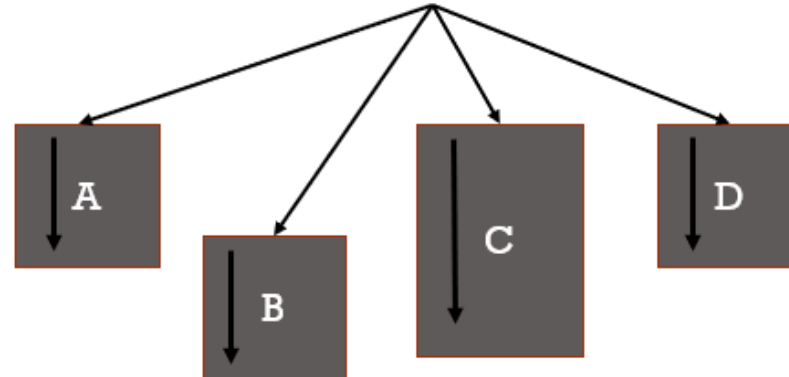
El modelo del proceso

Un contador de programa



Conmutación de procesos

Cuatro contadores de programa virtuales



Creación de un proceso

Los sistemas operativos necesitan cierta manera de crear procesos.

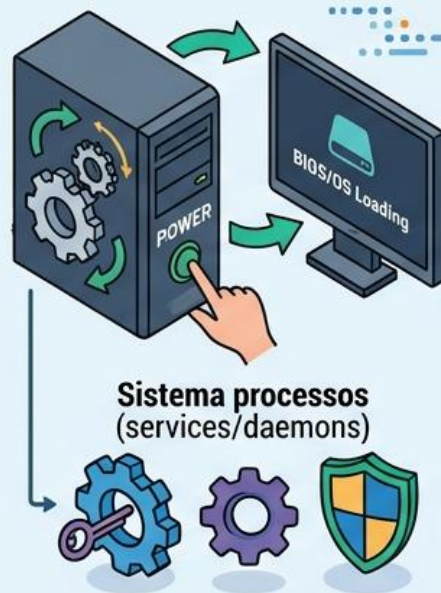
Hay **cuatro** eventos principales que provocan la creación de procesos:

1. El arranque del sistema.
2. La ejecución, desde un proceso, de una llamada al sistema para creación de procesos (tareas que se pueden dividir en procesos).
3. Una petición de usuario para crear un proceso.
4. El inicio de un trabajo por lotes.



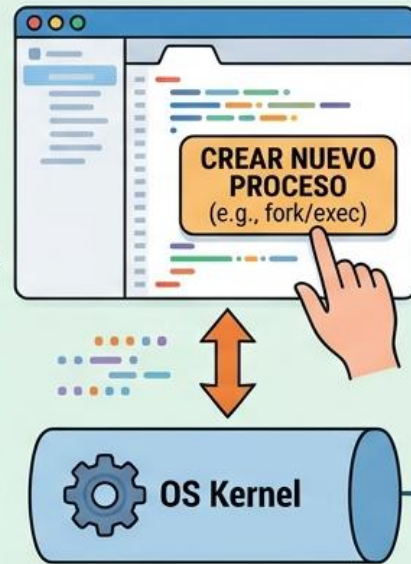
4 FORMAS DE CREACIÓN DE PROCESOS

1. INICIO DEL SISTEMA



Los procesos necesarios para el funcionamiento del SO se inician automáticamente al arrancar

2. LLAMADA AL SISTEMA DE CREACIÓN



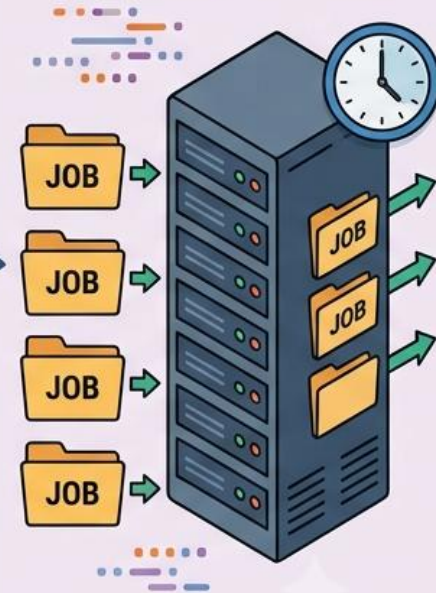
Un proceso en ejecución solicita explícitamente al Kernel crear un nuevo proceso hijo

3. PETICIÓN DEL USUARIO



El usuario inicia manualmente una aplicación a través de la interfaz gráfica o línea de comandos

4. INICIO DE UN TRABAJO POR LOTES



Un sistema de lotes (batch system) inicia automáticamente procesos programados secuencialmente

Creación de un proceso

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();

    printf("Hola desde un proceso\n");

    return 0;
}
```



Creación de un proceso

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();

    printf("Hola desde un proceso\n");

    return 0;
}
```

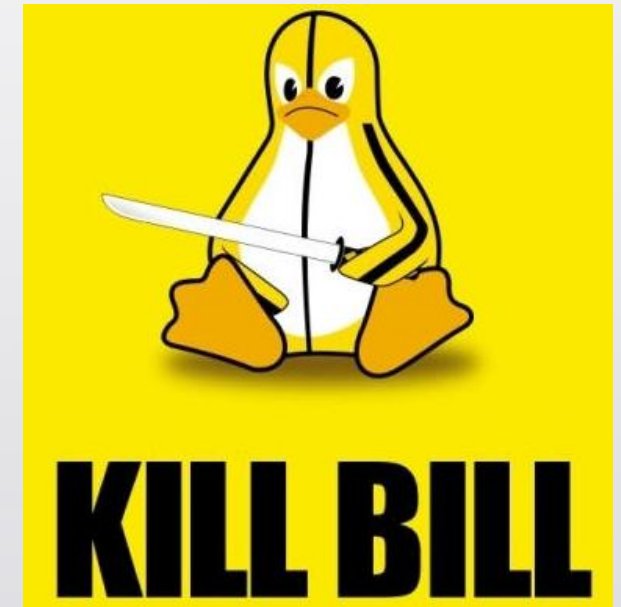
```
alumnado@debianS0:~$ nano ej1.c
alumnado@debianS0:~$ gcc ej1.c -o ej1
alumnado@debianS0:~$ ./ej1
Hola desde un proceso
Hola desde un proceso
```


Terminación de procesos

nada dura para siempre!!!

Los procesos terminan debido a las siguientes condiciones:

1. Salida normal (voluntaria).
2. Salida por error (voluntaria).
3. Error fatal (involuntaria).
4. Eliminado por otro proceso (involuntaria).

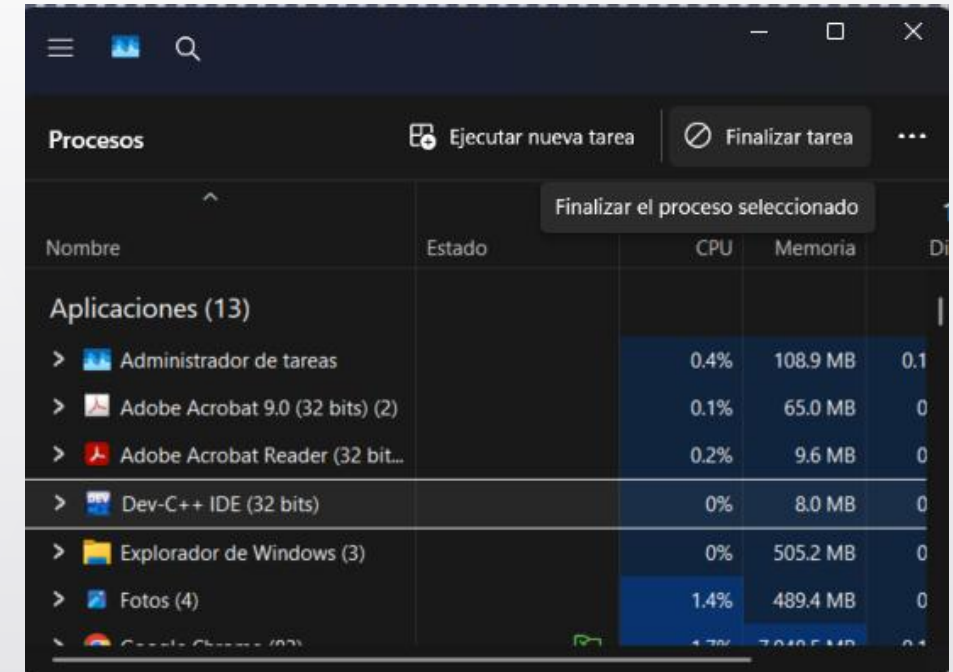


Terminación de procesos-Ejemplos

```
int main() {  
    printf("Saludos alumnos de SO\n");  
    return 0;  
}
```

```
int x = 10;  
int y = 0;  
int resultado = x / y;
```

```
FILE *archivo = fopen("datos.txt", "r");  
if (archivo == NULL) {  
    printf("Error Crucial: El archivo 'datos.txt' no existe.\n");  
    exit(1);  
}
```



CUATRO CAUSAS DE TERMINACIÓN DE PROCESOS

1. SALIDA VOLUNTARIA NORMAL



El proceso finaliza su tarea y se retira voluntariamente.

2. SALIDA VOLUNTARIA POR ERROR



El proceso detecta un error irreparable en tiempo de ejecución y decide terminar.

3. SALIDA INVOLUNTARIA POR ERROR



El Sistema Operativo fuerza la terminación por un error crítico del programa.

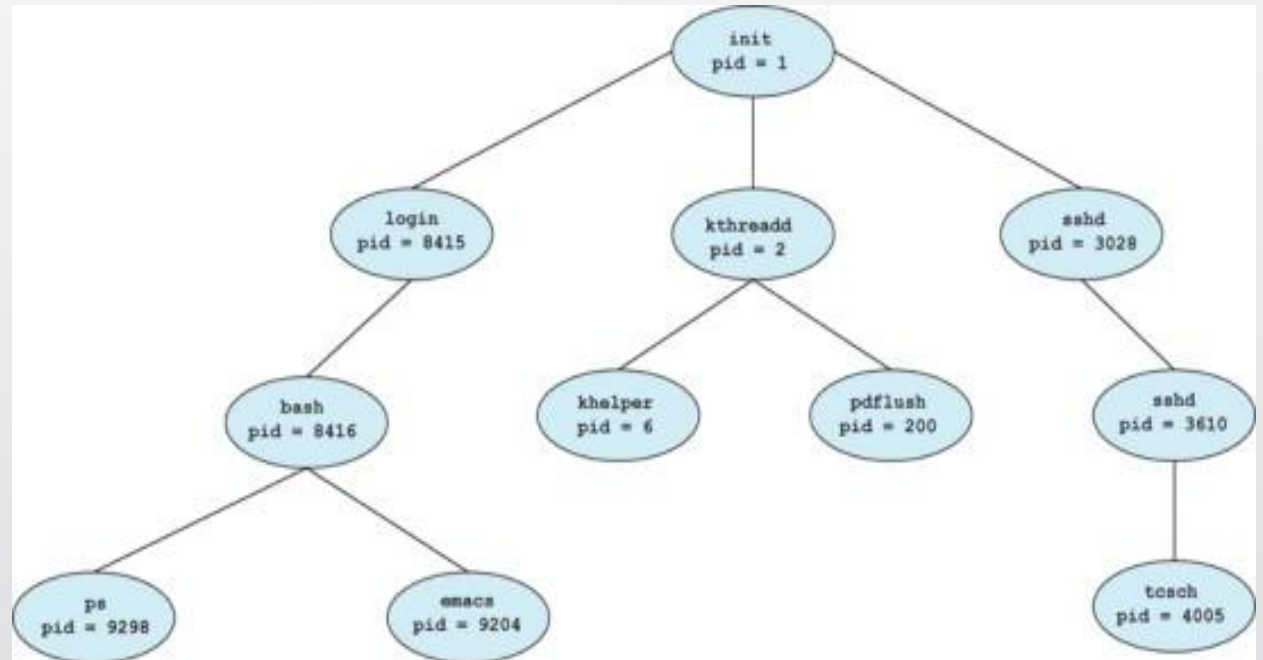
4. SALIDA INVOLUNTARIA POR OTRO PROCESO



Un proceso superior o el Administrador del Sistema (System Administrator) termina el proceso de forma forzosa (e.g., *kill command*).

Jerarquía de procesos

- Se refiere a la asociación de procesos
- Todos los procesos tienen un solo padre excepto la raíz
- Un proceso puede crear otros procesos, entonces un proceso padre también puede ser hijo de otro proceso (Jerarquía de procesos)
- En UNIX, un proceso y todos sus hijos forman un grupo de procesos
- En Windows no existe una jerarquía de procesos -> Todos son iguales -> Padre e hijo (Token)



Jerarquía de procesos

Proceso Padre

Cualquier proceso que ejecuta la llamada al sistema para crear uno nuevo. En UNIX, esto se realiza mediante la instrucción `fork()`.

El padre puede supervisar al hijo, esperar su terminación o heredarle recursos críticos.

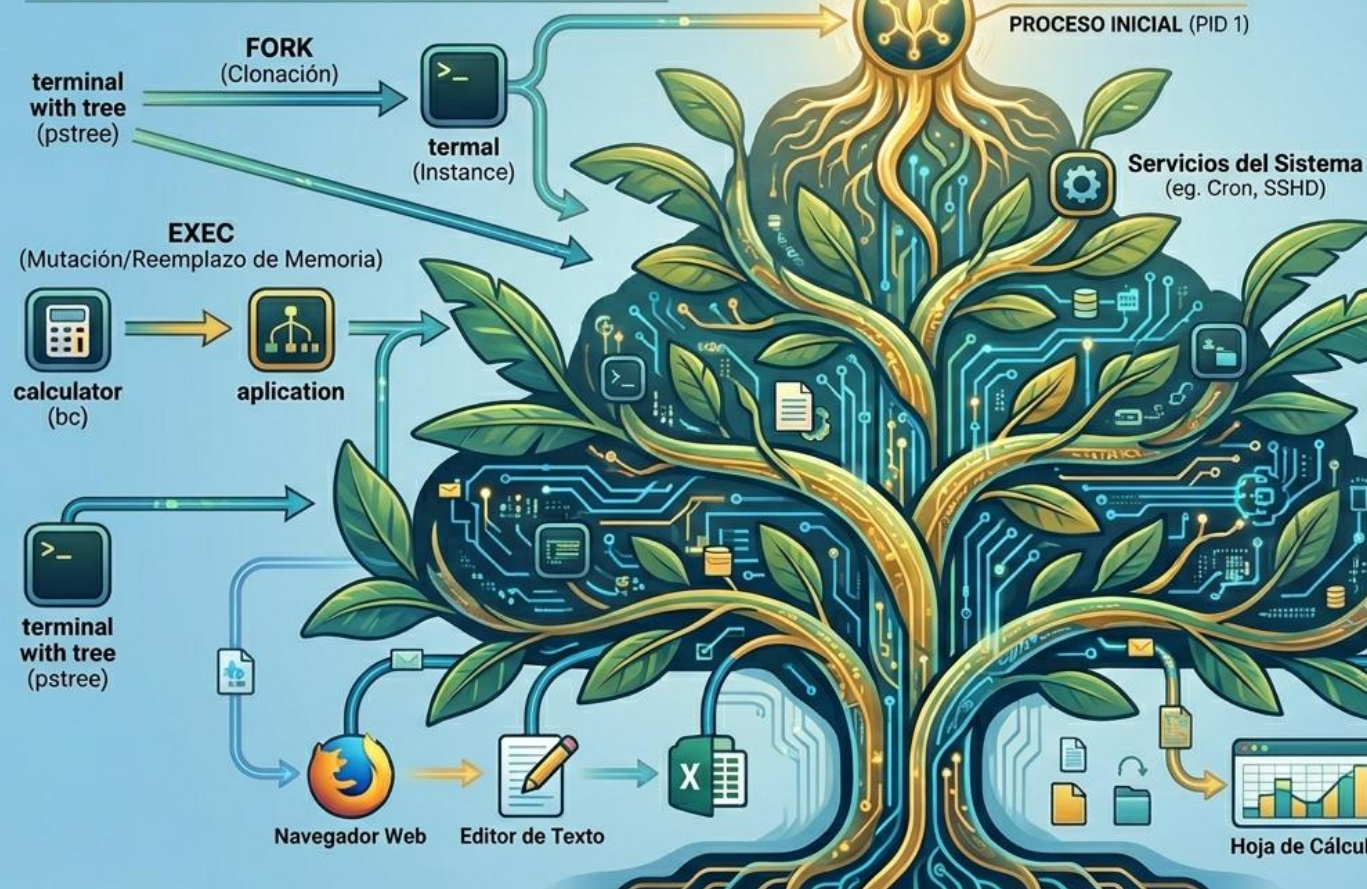
Proceso Hijo

El nuevo proceso creado. Es una copia exacta del padre al momento de nacer.

Posteriormente, puede mutar su código usando `exec()` para convertirse en una entidad independiente.

JERARQUÍA DE PROCESOS

ESTRUCTURA DE PROCESOS DEL SISTEMA



GESTIÓN DEL CICLO DE VIDA

ESPERA (WAIT)

Cascading nodes
conclonan persistente
poriaante en un árbol
persistente.

TERMINACIÓN (EXIT/KILL)

No cascading nodes
de handle-based y
no o persistente
norma.

Modelo UNIX	Modelo Windows
cascading nodes	handle-based
persistent árbe	no persistente

HERENCIA DE RECURSOS (Archivos, Variables)

GRUPOS DE PROCESOS Y SEÑALES (eg. Ctrl+C)



Jerarquía de un procesos

¿Cuál es la salida?

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid == 0)
    {
        printf("Soy el hijo\n");

        char *args[] = {"xfce4-terminal", NULL};

        execvp("xfce4-terminal", args);
    }
    else
    {
        printf("Soy el padre\n");
    }

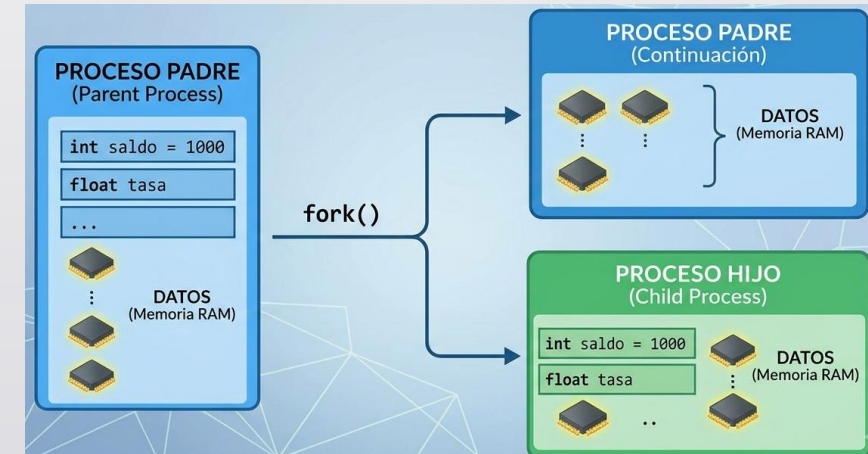
    return 0;
}
```

Herencia de datos en los procesos

Al momento de usar el **fork**, el hijo hereda **casi todo** lo que el padre tenía en ese milisegundo exacto:

1. **Los Descriptores de Archivos Abiertos:** Archivo de texto, una conexión de red o un canal de comunicación
2. **El Espacio de Memoria (Datos, Stack y Heap):** El hijo obtiene una copia idéntica de todas las variables, arreglos y datos que el padre tenía en la memoria RAM hasta ese momento.
3. **El Directorio de Trabajo Actual:** Si el padre estaba ejecutándose dentro de /home/usuario/documentos, el hijo nacerá operando exactamente en esa misma carpeta.
4. **Uid y Gid (Credenciales de seguridad):** El hijo hereda los mismos privilegios de usuario (UID) y de grupo (GID) del padre.

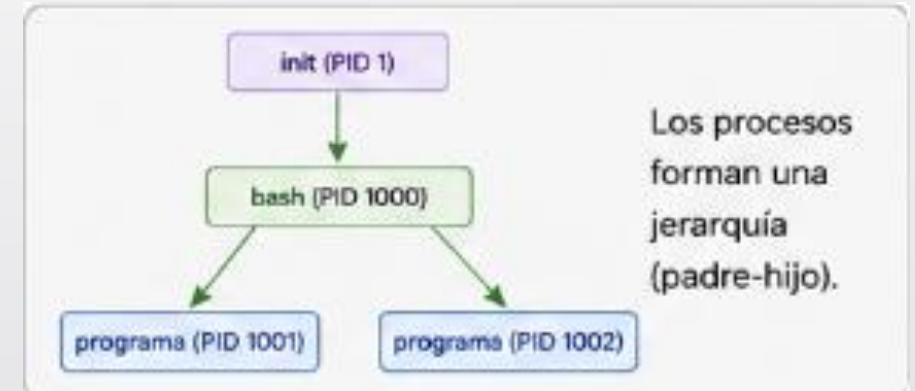
Descriptor	Recurso abierto
0	stdin (teclado)
1	stdout (pantalla)
2	stderr (pantalla)
3	archivo.txt
4	datos.bin
...	...



Herencia de datos en los procesos

Algunos puntos clave en la herencia de procesos son:

1. Los identificadores de los procesos no se heredan (**PID**) ¿Y los **PPID**?
2. **Independencia Absoluta (Aislamiento):** Aunque las variables se llamen igual y tengan el mismo valor inicial, están en direcciones de memoria física distintas. **No comparten memoria.**
3. **Modificaciones:** Cualquier cambio que el hijo haga en sus datos locales no afectará en absoluto a las variables del proceso padre, y viceversa.



Prueba el siguiente programa en **Debian**

¿Qué hace?

¿Cuál es la salida?

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    int x = 10;

    FILE *archivo;

    archivo = fopen("salida.txt", "a");

    if(archivo == NULL){
        printf("Error al abrir archivo\n");
        return 1;
    }

    pid_t pid = fork();

    if(pid == 0){
        x = 100;

        printf("\n____Hijo____\n");
        printf("PID   : %d\n", getpid());
        printf("PPID  : %d\n", getppid());
        printf("x     : %d\n", x);

        fprintf(archivo, "Hijo: PID=%d PPID=%d x=%d\n", getpid(), getppid(), x);
    }else{

        x = 50;
        printf("\n____Padre____\n");
        printf("PID   : %d\n", getpid());
        printf("PPID  : %d\n", getppid());
        printf("x     : %d\n", x);

        fprintf(archivo, "PADRE: PID=%d PPID=%d x=%d\n", getpid(), getppid(), x);
    }

    fclose(archivo);

    return 0;
}
```

Prueba el siguiente programa en Debian

¿Qué hace?

¿Cuál es la salida?

```
alumnado@debianSO:~/Documentos/programas$ ./herencia

-----Padre-----
PID   : 1789
PPID  : 1731
x     : 50

-----Hijo-----
PID   : 1790
PPID  : 1789
x     : 100
alumnado@debianSO:~/Documentos/programas$ ls
herencia herencia.c salida.txt sumasInfinitas sumasInfinitas.c
alumnado@debianSO:~/Documentos/programas$ nano salida.txt
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    int x = 10;

    FILE *archivo;

    archivo = fopen("salida.txt", "a");

    if(archivo == NULL){
        printf("Error al abrir archivo\n");
        return 1;
    }

    pid_t pid = fork();

    if(pid == 0){
        x = 100;

        printf("\n-----Hijo-----\n");
        printf("PID   : %d\n", getpid());
        printf("PPID  : %d\n", getppid());
        printf("x     : %d\n", x);

        fprintf(archivo, "Hijo: PID=%d PPID=%d x=%d\n", getpid(), getppid(), x);
    }else{

        x = 50;
        printf("\n-----Padre-----\n");
        printf("PID   : %d\n", getpid());
        printf("PPID  : %d\n", getppid());
        printf("x     : %d\n", x);

        fprintf(archivo, "PADRE: PID=%d PPID=%d x=%d\n", getpid(), getppid(), x);
    }

    fclose(archivo);

    return 0;
}
```



Proceso: Estados de un proceso

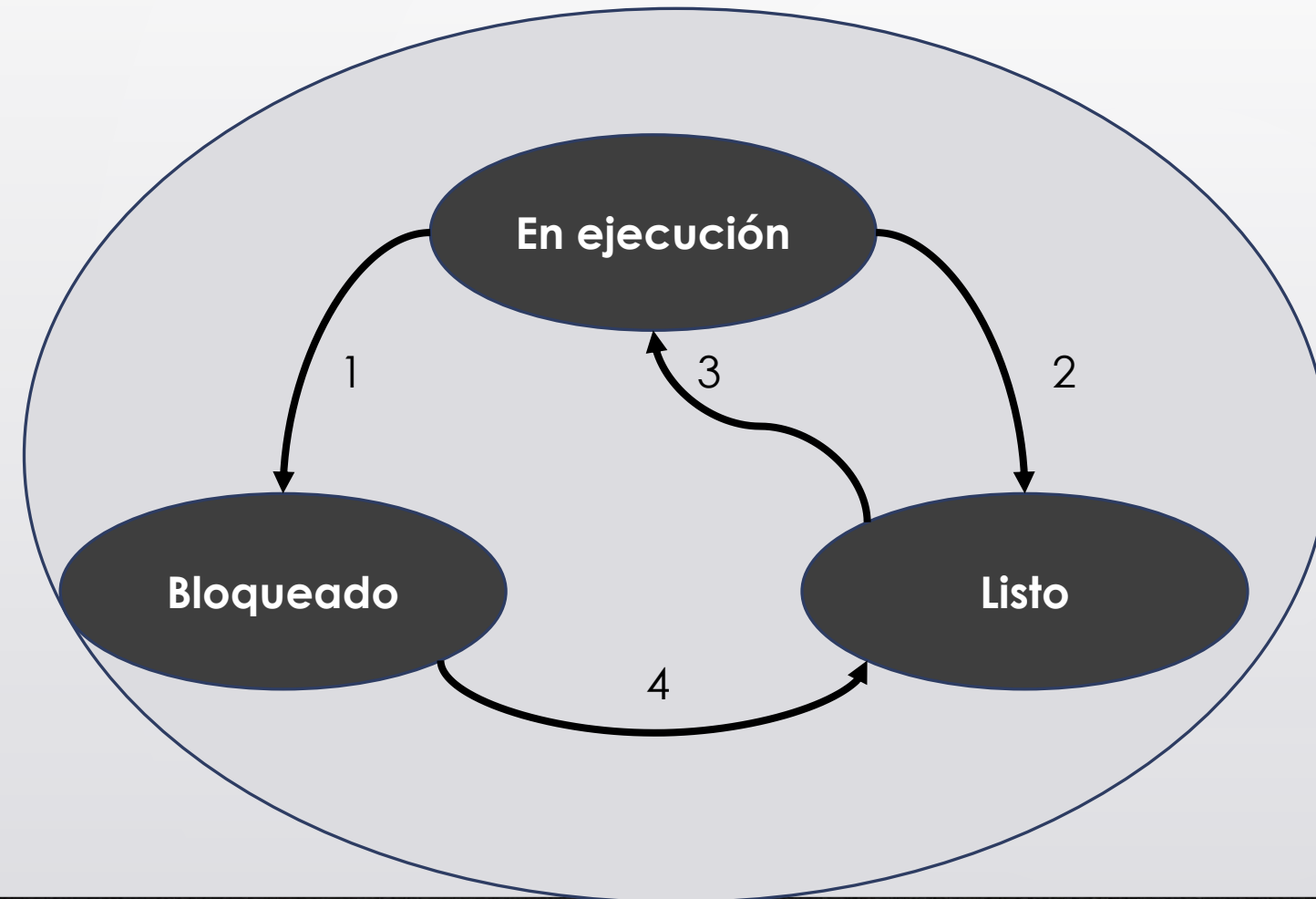
- Existen tres estados en los que puede estar un proceso:

1. En ejecución -> Utilizando la CPU
2. Listo - > Ejecutable, esperando a utilizar la CPU
3. Bloqueado -> No se puede ejecutar sino hasta que ocurra cierto evento, aun cuando la CPU esté disponible



Similares

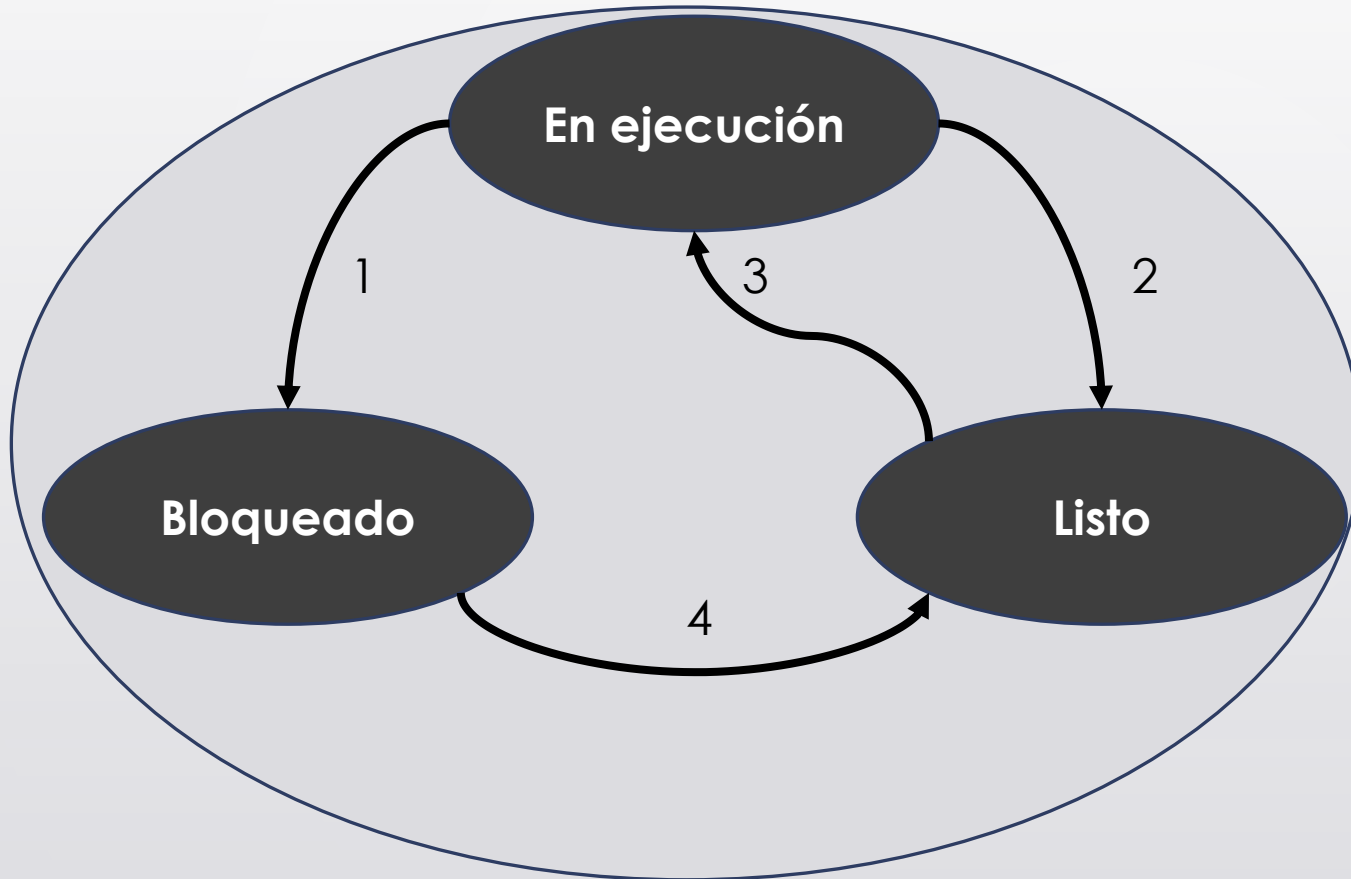
Proceso: Estados de un proceso



1. El proceso se bloquea para recibir entrada
2. El CPU selecciona otro proceso
3. El CPU selecciona este proceso
4. La entrada ya está disponible

Fig. Diagrama de estados de un proceso.

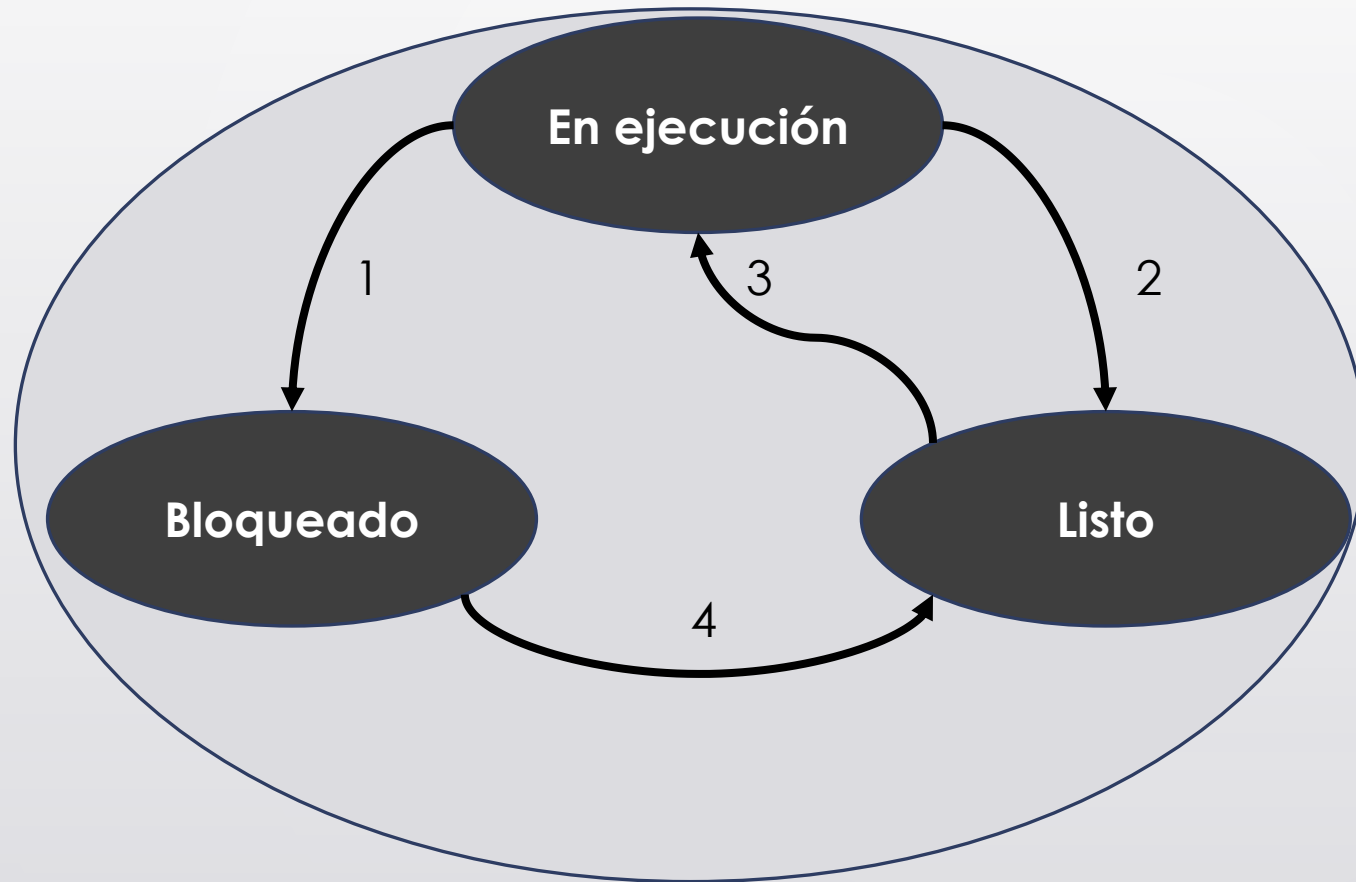
Proceso: Estados de un proceso



1. El proceso se bloquea debido a que está esperando algún **evento** necesario para continuar con su ejecución

Fig. Diagrama de estados de un proceso.

Proceso: Estados de un proceso



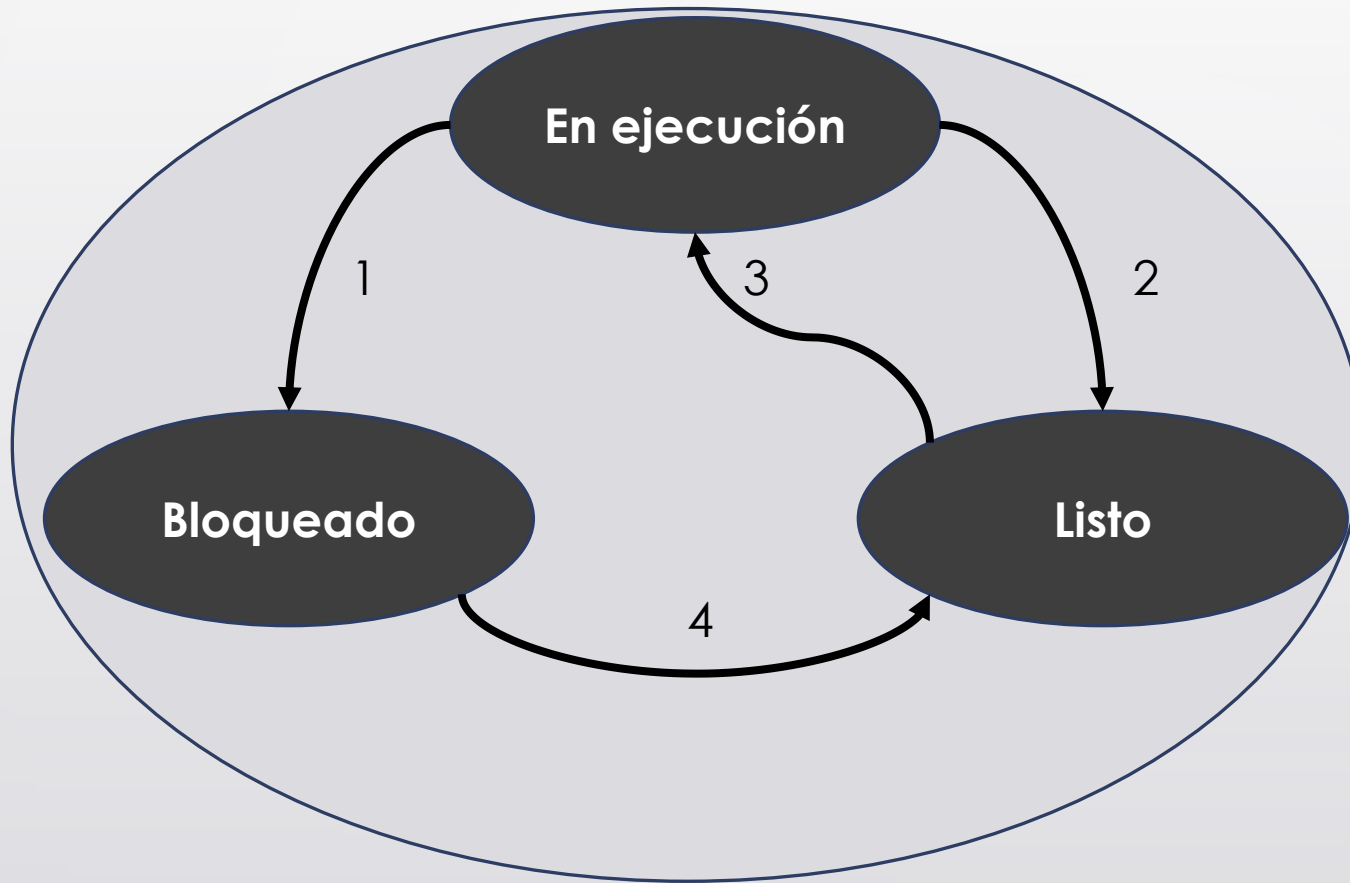
2 y 3. Se producen por una parte del SO conocida como **planificador**

2. El **planificador** determina que el proceso actual se ha ejecutado tiempo suficiente en la CPU

3. El **planificador** determina que el proceso actual obtenga la CPU

Fig. Diagrama de estados de un proceso.

Proceso: Estados de un proceso

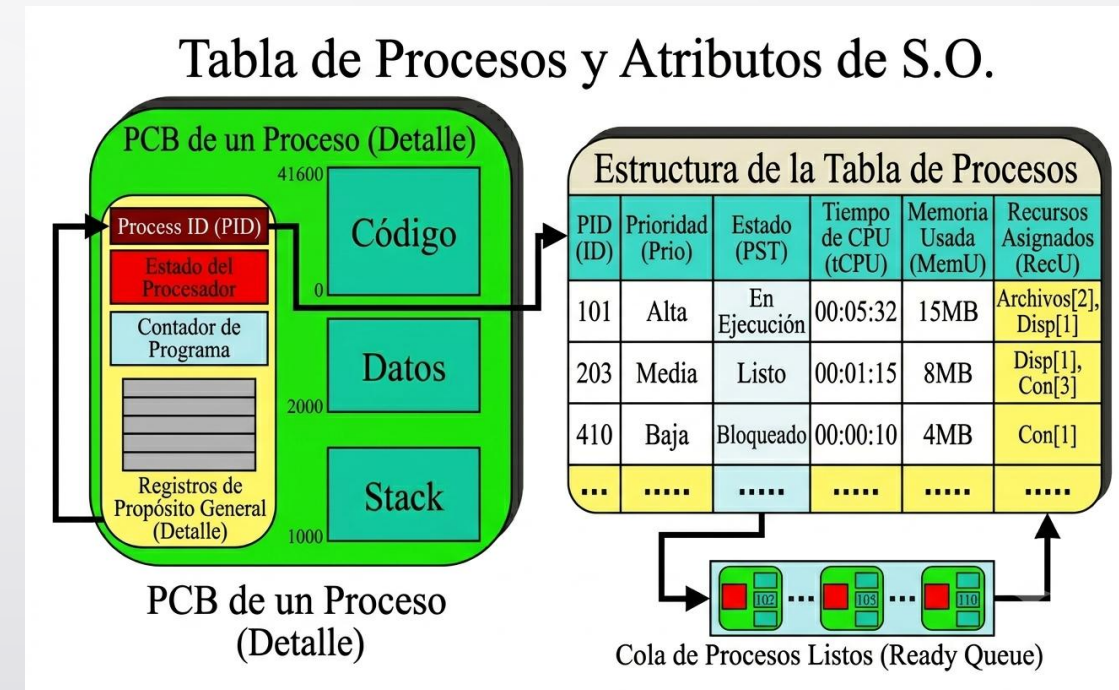


4. Se produce el **evento** externo esperado por el proceso. En caso de que no exista otro proceso, se activa la transición 3

Fig. Diagrama de estados de un proceso.

Tabla de Procesos

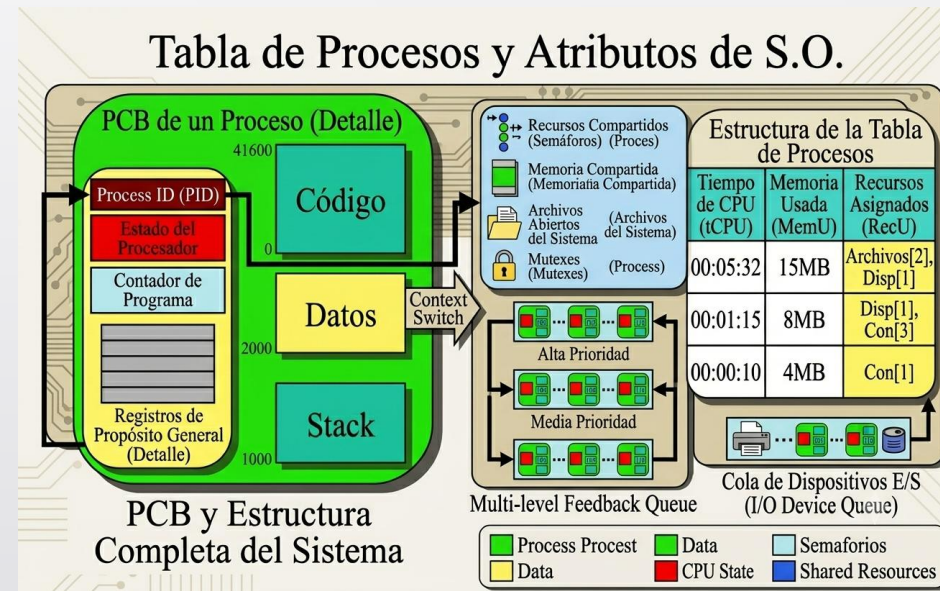
- Es una de las estructuras de datos más importantes del **kernel** (núcleo) del SO.
- Es un **arreglo** donde se guarda **toda la información** de cada proceso que está en el sistema en ese momento.
- Es necesaria para retomar la ejecución de un proceso después de una interrupción.
- Cada **fila** de esta tabla se llama **PCB** (Process Control Block o Bloque de Control de Proceso).
- Sin esta tabla, la multiprogramación sería imposible.



Atributos esenciales que componen un PCB

Gestión de Procesos (Identificación y Estado)

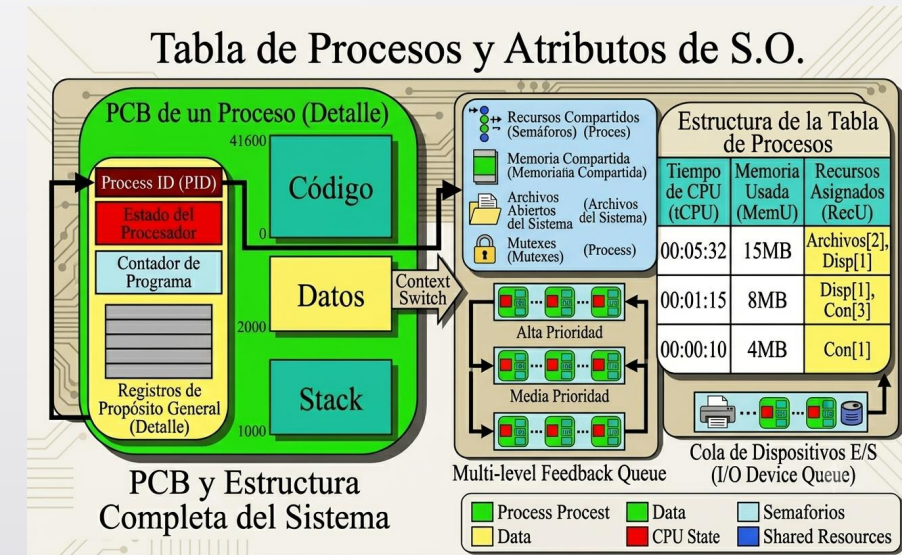
- **PID (Process ID):** El número de identificación único que el Kernel le asigna al proceso al nacer.
- **PPID (Parent Process ID):** El identificador del proceso padre que lo creó.
- **Estado del proceso:** Indica si está en ejecución, listo o bloqueado.
- **Contador de Programa (PC):** Contiene la dirección de memoria de la siguiente instrucción que el procesador debe ejecutar cuando el proceso vuelva a la vida.
- **Registros de la CPU:** Copia de todos los registros de hardware del procesador (como el acumulador, punteros de pila, etc.) acumulados en el instante exacto en que el proceso fue pausado.
- **Prioridad:** El nivel de importancia que tiene para el planificador (Scheduler).



Atributos esenciales que componen un PCB

Gestión de Memoria (Espacio Operativo)

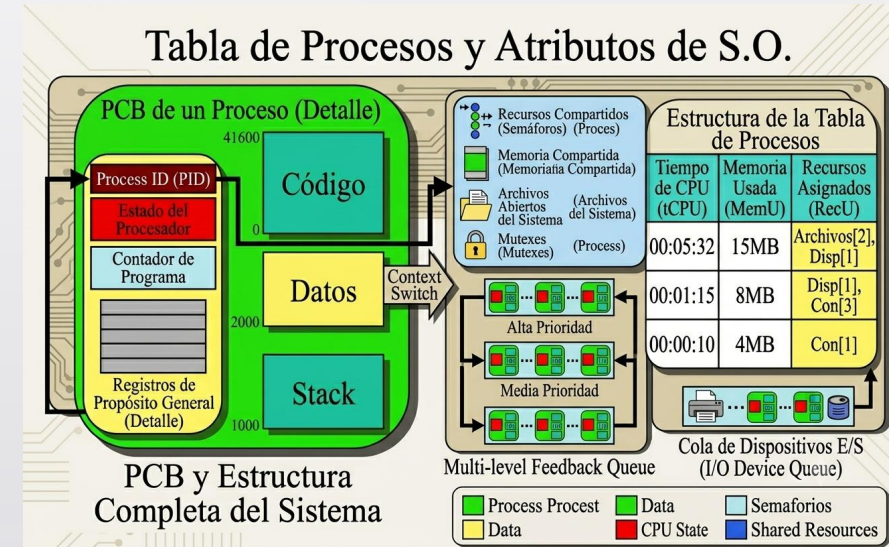
- **Apuntadores a segmentos:** Direcciones que delimitan dónde empieza y dónde termina el código, los datos, el Stack (pila) y el Heap (memoria dinámica) del proceso en la memoria RAM.
- **Tablas de paginación:** Información indispensable si el sistema operativo utiliza memoria virtual (paginación) para traducir direcciones lógicas a físicas.



Atributos esenciales que componen un PCB

Gestión de Archivos y Recursos (Entrada/Salida)

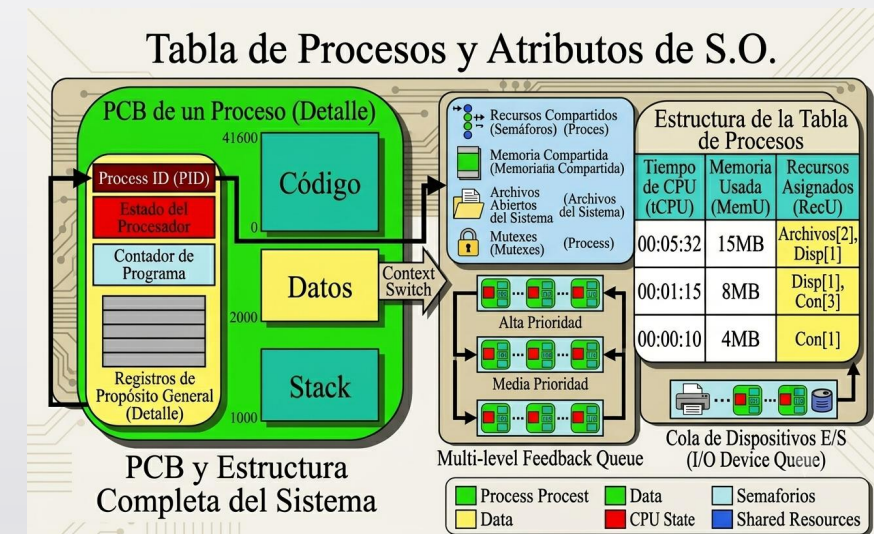
- **Tabla de Descriptores de Archivos:** Una lista de todos los archivos o conexiones de red que el proceso tiene abiertos en ese momento.
- **Directorio de trabajo:** La ruta de la carpeta actual desde donde opera el proceso.
- **Propietario (UID/GID):** Identificación del usuario y grupo que ejecutó el proceso, lo que determina sus permisos de seguridad.



Atributos esenciales que componen un PCB

Gestión de Archivos y Recursos (Entrada/Salida)

- **Tabla de Descriptores de Archivos:** Una lista de todos los archivos o conexiones de red que el proceso tiene abiertos en ese momento.
- **Directorio de trabajo:** La ruta de la carpeta actual desde donde opera el proceso.
- **Propietario (UID/GID):** Identificación del usuario y grupo que ejecutó el proceso, lo que determina sus permisos de seguridad.





Atributos que verás en Linux

Atributo	Descripción	Comando para ver
PID	Identihcador único del proceso	ps, top
%»CPU / %»MEM	Uso actual de recursos	top, htop
TTY	Terminal asociada al proceso	ps aux
STAT	Estado actual (R, S, D, Z, T)	ps -l
PRI / NI	Prioridad y valor "nice" (cortesía)	top



Estados de un proceso en linux

Código	Estado	Significado
R	Running	El proceso está ejecutándose o listo para ejecutarse en la CPU.
S	Sleeping	El proceso está esperando algún evento (teclado, red, temporizador, etc.). Es el estado más común.
D	Disk Sleep	Espera ininterrumpible, normalmente esperando una operación de entrada/salida (disco, dispositivo, etc.).
T	Stopped	El proceso está detenido o suspendido.
Z	Zombie	El proceso ya terminó pero aún existe una entrada en la tabla de procesos.
I	Idle	Hilo del kernel inactivo (aparece en Linux modernos).

Prueba el siguiente programa en **Debian**

1. Ejecútalo en una terminal
2. En otra terminal ejecuta htop
3. Busca el proceso con F3 o fíltralo con F4
4. Busca la siguiente información:

Columna	Nombre completo	¿Qué significa?
PID	Process ID	Identificador único del proceso.
USER	User	Usuario propietario del proceso.
PRI	Priority	Prioridad asignada por el sistema operativo.
NI	Nice Value	Valor que modifica la prioridad del proceso.
VIRT	Virtual Memory	Memoria virtual total asignada al proceso.
RES	Resident Memory	Memoria RAM que está utilizando actualmente.
SHR	Shared Memory	Memoria compartida con otros procesos.
S	State	Estado actual del proceso.
CPU%	CPU Usage	Porcentaje de CPU utilizado por el proceso.
MEM%	Memory Usage	Porcentaje de memoria RAM utilizada por el proceso.
TIME+	CPU Time	Tiempo acumulado de CPU consumido desde que inició.
Command	Command	Nombre del programa o comando ejecutado.

- 5.-Investiga que valores puede tener “S” y observa si cambia a lo largo de esta ejecución
¿En qué momento?

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    unsigned long long contador = 0;

    printf("PID = %d\n", getpid());
    printf("PPID = %d\n", getppid());

    printf("\nFase 1: el proceso dormira 30 segundos.\n");
    sleep(30);

    printf("\nFase 2: el proceso utilizara CPU durante 60 segundos.\n");

    for(int segundos = 0; segundos < 60; segundos++)
    {
        for(unsigned long long i = 0; i < 500000000ULL; i++)
        {
            contador++;
        }
        printf("Segundo %d: contador = %llu\n", segundos + 1, contador);
    }

    printf("\nProceso terminado.\n");

    return 0;
}
```


Prueba el siguiente programa en **Debian**

1. Ejecútalo en una terminal
2. En otro terminal ejecuta htop
3. Busca el proceso con F3 o fíltralo con F4
4. Busca la siguiente información:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    unsigned long long contador = 0;

    printf("PID = %d\n", getpid());
    printf("PPID = %d\n", getppid());

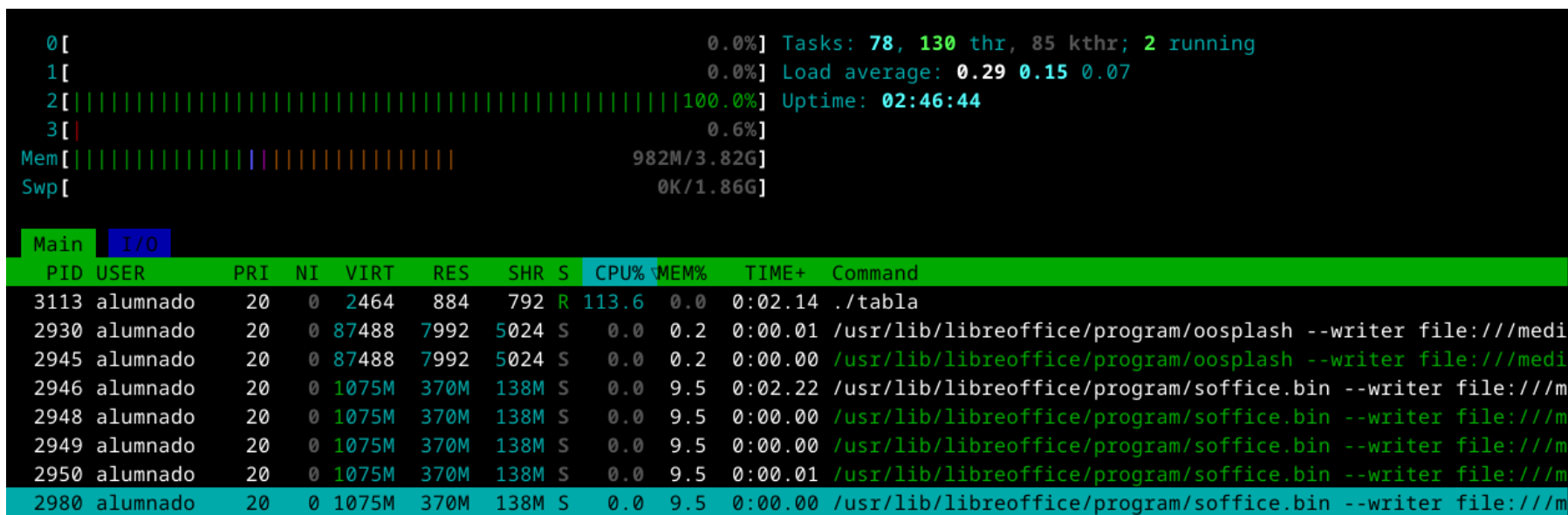
    printf("\nFase 1: el proceso dormira 30 segundos.\n");
    sleep(30);

    printf("\nFase 2: el proceso utilizara CPU durante 60 segundos.\n");

    for(int segundos = 0; segundos < 60; segundos++)
    {
        for(unsigned long long i = 0; i < 500000000ULL; i++)
        {
            contador++;
        }
        printf("Segundo %d: contador = %llu\n", segundos + 1, contador);
    }

    printf("\nProceso terminado.\n");

    return 0;
}
```



The screenshot shows the htop interface. At the top, system statistics are displayed: 0.0% CPU usage, 78 tasks, 130 threads, 85 kthreads, 2 running processes, 0.29/0.15/0.07 load average, 02:46:44 uptime, 0.6% memory usage, 982M/3.82G memory usage, and 0K/1.86G swap usage. Below the statistics, there are tabs for 'Main' and 'I/O'. The 'Main' tab is selected, showing a table of processes. The table has columns for PID, USER, PRI, NI, VIRT, RES, SHR, S, CPU%, MEM%, TIME+, and Command. The process list shows several instances of 'alumnado' and 'soffice.bin'.

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
3113	alumnado	20	0	2464	884	792	R	113.6	0.0	0:02.14	./tabla
2930	alumnado	20	0	87488	7992	5024	S	0.0	0.2	0:00.01	/usr/lib/libreoffice/program/oosplash --writer file:///medi
2945	alumnado	20	0	87488	7992	5024	S	0.0	0.2	0:00.00	/usr/lib/libreoffice/program/oosplash --writer file:///medi
2946	alumnado	20	0	1075M	370M	138M	S	0.0	9.5	0:02.22	/usr/lib/libreoffice/program/soffice.bin --writer file:///m
2948	alumnado	20	0	1075M	370M	138M	S	0.0	9.5	0:00.00	/usr/lib/libreoffice/program/soffice.bin --writer file:///m
2949	alumnado	20	0	1075M	370M	138M	S	0.0	9.5	0:00.00	/usr/lib/libreoffice/program/soffice.bin --writer file:///m
2950	alumnado	20	0	1075M	370M	138M	S	0.0	9.5	0:00.01	/usr/lib/libreoffice/program/soffice.bin --writer file:///m
2980	alumnado	20	0	1075M	370M	138M	S	0.0	9.5	0:00.00	/usr/lib/libreoffice/program/soffice.bin --writer file:///m

UNIX: LLAMADAS AL SISTEMA

getpid() es una llamada al sistema en UNIX que devuelve el **ID del proceso** (PID) que la ejecuta.

Para la creación de procesos se tienen las llamadas

- **fork()** : Permite clonar un proceso a partir de uno existente
- **execve()**: Permite ejecutar un programa o proceso con distintos parámetros

```
#include <stdio.h>
int main() {
    int id = getpid();
    printf("Mi id es: %d\n", id);
    return 0;
}
```

```
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        execl("/bin/echo", "echo", "Hola desde el proceso hijo", NULL);
    }

    return 0;
}
```

UNIX: LLAMADAS AL SISTEMA

fork() es una llamada al sistema en UNIX que crea un nuevo proceso duplicando el proceso que la invoca.

Funcionamiento:

- **Proceso hijo:** Es una copia casi exacta del proceso padre, con su propio espacio de memoria. Inicia su ejecución desde la línea posterior en donde fue creado.
- **Valores de retorno:**
 - 0: Retornado al proceso hijo.
 - >0: Retornado al proceso padre, representa el PID del hijo.
 - -1: Indica que la creación del proceso hijo falló.

```
#include <sys/types.h>
#include <unistd.h>

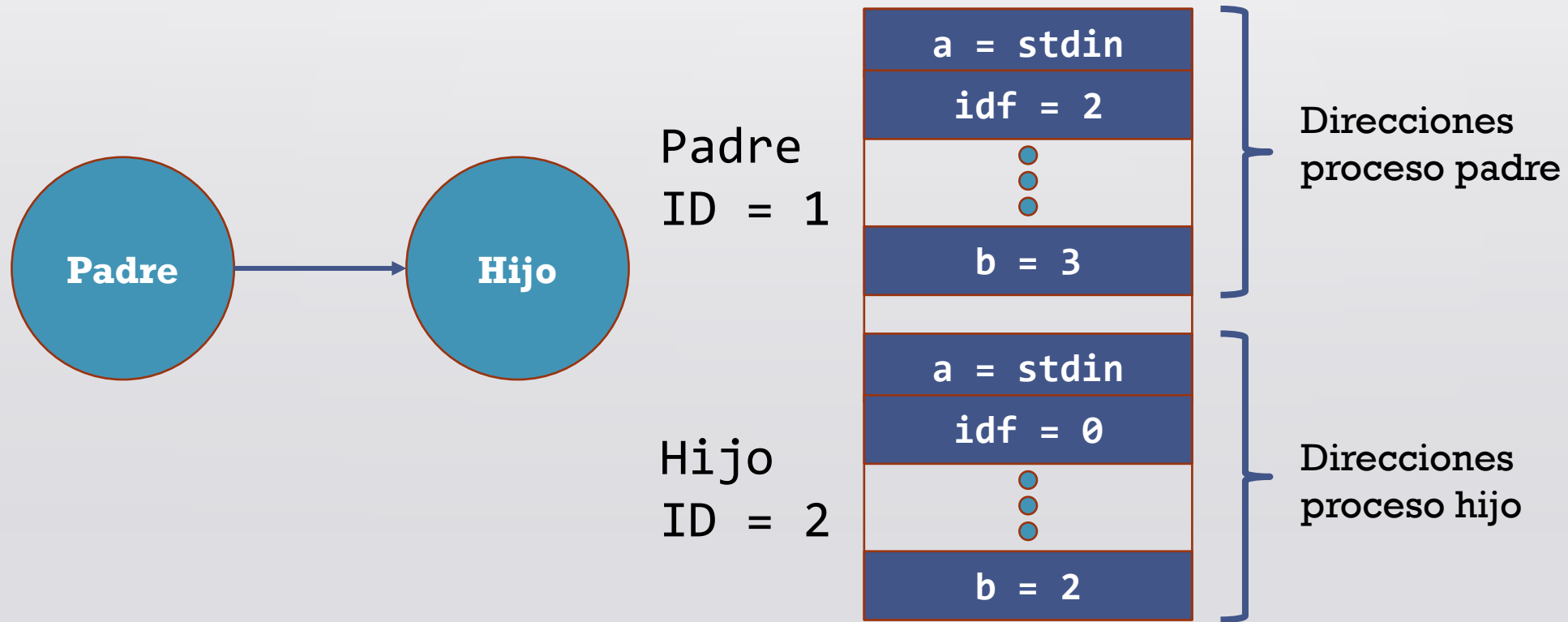
int main() {
    pid_t idf = fork();

    if (idf == 0) {
        // hijo
    } else if (idf > 0) {
        // padre
    } else {
        // error
    }
}
```

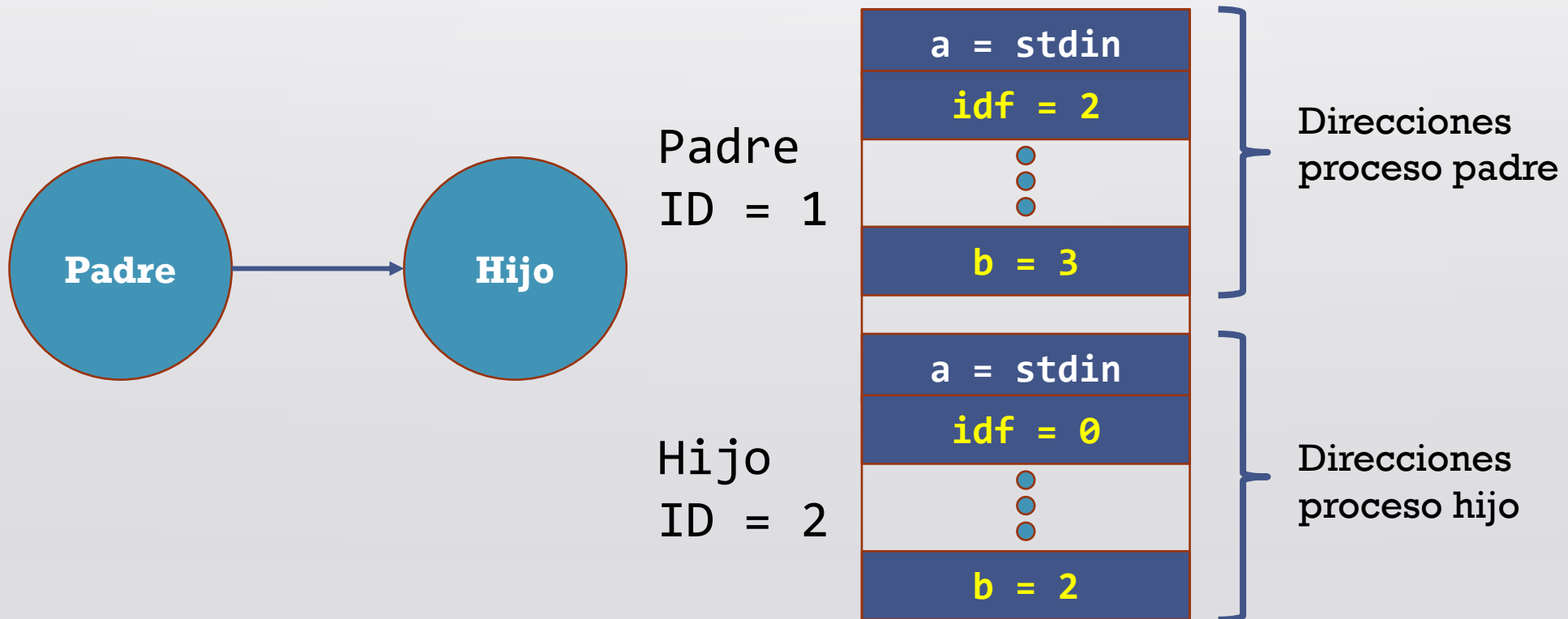
```
#include <stdio.h>
#include <unistd.h>

int main() {
    int a, b = 3;
    scanf("%d", &a);
    pid_t idf = fork();
    if(idf == 0) {
        b = 2;
    }
    printf("%d\n", a + b);
    return 0;
}
```


UNIX: PROCESOS



UNIX: PROCESOS



UNIX: LLAMADAS AL SISTEMA

Escriba, ejecute y responda las siguientes preguntas:

- ¿Para qué sirve el condicional `if(idf == 0)` en el código?
- ¿Qué proceso se ejecutará primero?
¿Siempre será así?

Añada una instrucción para que el programa no finalice.

- ¿Cómo se visualiza con la instrucción **ps tree** el árbol de procesos en la ejecución del programa?

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int id = getpid();
    printf("Soy el proceso padre\n");
    pid_t idf = fork();
    if(idf == 0) {
        printf("Soy el proceso hijo\n");
        printf("Mi papa es el proceso con ID: %d\n", id);
        id = getpid();
        printf("Mi ID es: %d\n\n", id);
    } else {
        printf("Mi hijo es el proceso con ID: %d\n", idf);
        printf("Mi ID es: %d\n\n", id);
    }
    return 0;
}
```


UNIX: LLAMADAS AL SISTEMA

- ¿Cómo se visualiza con la instrucción **pstree** el árbol de procesos en la ejecución del programa?

```
└─xfce4-terminal─┬─bash──fork3──fork3
                  │
                  └─bash──pstree
                      3*[{xfce4-terminal}]
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int id = getpid();
    printf("Soy el proceso padre\n");
    pid_t idf = fork();
    if(idf == 0) {
        printf("Soy el proceso hijo\n");
        printf("Mi papa es el proceso con ID: %d\n", id);
        id = getpid();
        printf("Mi ID es: %d\n\n", id);

        sleep(10);
        printf("El hijo ha terminado\n");
    } else {
        printf("Mi hijo es el proceso con ID: %d\n", idf);
        printf("Mi ID es: %d\n\n", id);
        sleep(12);

        wait(NULL);
        printf("El padre ha terminado\n");
    }
    return 0;
}
```

UNIX: Proceso Zombie

- Es un proceso hijo que ya terminó su ejecución, pero cuya entrada permanece temporalmente en la tabla de procesos porque el proceso padre aún no ha recuperado su estado de terminación mediante wait().
- wait() hace que el proceso padre espere a que finalice un proceso hijo.

Main	I/O										
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
6236	alumnado	20	0	2464	852	764	S	0.0	0.0	0:00.00	./zombie
6237	alumnado	20	0	0	0	0	Z	0.0	0.0	0:00.00	zombie

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int id = getpid();
    printf("Soy el proceso padre\n");
    pid_t idf = fork();
    if(idf == 0) {
        printf("Soy el proceso hijo\n");
        printf("Mi papa es el proceso con ID: %d\n", id);
        id = getpid();
        printf("Mi ID es: %d\n\n", id);

        sleep(2);
        printf("El hijo ha terminado\n");
    } else {
        printf("Mi hijo es el proceso con ID: %d\n", idf);
        printf("Mi ID es: %d\n\n", id);

        sleep(60);

        wait(NULL);
        printf("El padre ha terminado\n");
    }
    return 0;
}
```

UNIX: Proceso Zombie Características

- El proceso ya terminó.
- No ejecuta instrucciones.
- No consume CPU.
- Conserva su PID y estado de salida.
- Permanece en la tabla de procesos.
- Desaparece cuando el padre ejecuta wait().

Main	I/O											
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command	
6236	alumnado	20	0	2464	852	764	S	0.0	0.0	0:00.00	./zombie	
6237	alumnado	20	0	0	0	0	Z	0.0	0.0	0:00.00	zombie	

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int id = getpid();
    printf("Soy el proceso padre\n");
    pid_t idf = fork();
    if(idf == 0) {
        printf("Soy el proceso hijo\n");
        printf("Mi papa es el proceso con ID: %d\n", id);
        id = getpid();
        printf("Mi ID es: %d\n\n", id);

        sleep(2);
        printf("El hijo ha terminado\n");
    } else {
        printf("Mi hijo es el proceso con ID: %d\n", idf);
        printf("Mi ID es: %d\n\n", id);

        sleep(60);

        wait(NULL);
        printf("El padre ha terminado\n");
    }
    return 0;
}
```

UNIX: Procesos

- La llamada al sistema `fork()` crea un nuevo proceso hijo a partir de un proceso padre.
 - Si todos los procesos generados ejecutan cada llamada a `fork()`, el número total de procesos se duplica en cada ejecución.
 - Es decir la llamada **`fork()`** generaría 2^i procesos, donde i corresponde al número de veces que se ha ejecutado la instrucción **`fork()`**
 - Por ejemplo, el siguiente código generaría 4 procesos
- Es posible condicionar la creación de hijos? ¿Cómo?
 - ¿Se puede crear cualquier estructura?

...

```
idf = fork();  
idf = fork();
```

...

Actividad en clase

- Genere las siguientes estructuras
- Obligue a que cada proceso imprima su id, el de su padre y dormir un tiempo igual a 15s

